

A Practical Oracle-Free Preimage Attack on the 19-Round SHA-256 Compression Function via Schedule-Differential Table Lookup

Kameldip Singh Basra

Independent Researcher

kameldipbasra@gmail.com

ORCID: 0009-0001-0977-4614

June 2026

Abstract

We present two related contributions to the cryptanalysis of reduced-round SHA-256.

(1) Preimage attack. We give a practical oracle-free preimage attack on 19-round SHA-256. Given only a 32-byte target hash, the attack combines a backward-chain inversion with a precomputed 16 GB $\sigma_0(u) - u$ differential table and a two-stage fixed-point iteration, performing $O(2^{32})$ work per context attempt. Deduplicated measurements show that both halves of the W_9 filter carry substantial structural bias, well above the uniform 2^{-16} rate. A preregistered campaign of 240 fully attacked contexts on one H100 yields the first direct estimate of the full-match rate, $\hat{\mu} = 3/240 = 1.25 \times 10^{-2}$ per context (95% Poisson CI $[2.6 \times 10^{-3}, 3.7 \times 10^{-2}]$), which falls inside the independence-derived projection of 6×10^{-3} – 1.3×10^{-2} made before it was measured. Three events is a wide interval and we treat it accordingly; the marginal rates and their uncertainties are reported in §5.6. Three end-to-end attacks found verified preimages after 3, 20 and 34 contexts, which we present as demonstrations of feasibility rather than as a runtime estimator; the campaign above recovered three further preimages, for six independently verified in total. We further show that contexts are far from interchangeable and that a cheap 2^{20} screening pass—1/4096 of a context’s budget—selects productive ones: in a preregistered paired campaign (40 targets, 240 fully attacked contexts, matched 2^{32} exposure per arm) screening raised W_9 lo-pass yield per context by $3.41 \times$ (95% CI $[2.13, 5.82]$, target-blocked bootstrap; sign test $p = 3.4 \times 10^{-4}$). The effect is heterogeneous—it helped on 31 of 40 targets and hurt on 9—and its conversion through the final 32-bit filter to preimages remains unmeasured. All three were verified on a single NVIDIA H100 GPU in under 15 minutes each (cost $< \$1.50$). For context on the practical frontier: the most recent SAT-based analysis of round-reduced SHA-256 [8] finds full-target preimages at 16, 17 and 18 rounds but reports the 19-round problem *unsolved* under all four CNF encodings it tries, and—after a 24-hour cube-and-conquer run on 16 cores—concludes that a 19-round preimage is “too difficult a task even if one carefully selects the SAT encoding and uses parallel computing,” reaching 19 rounds only in a weakened form with at most 31 of the 256 output bits constrained. The present work produces three full-target 19-round preimages, each in minutes on a single GPU. The distinction is one of method rather than resources: dedicated algebraic analysis reaches a point that general-purpose search does not.

(2) Nine-Step Cancellation Lemma and partial collision. We prove a new algebraic identity: for *any* message block $W_0, \dots, W_{15}$ and any nonzero δ , the choice $\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$ forces $\Delta W_{16} = \dots = \Delta W_{23} = 0$ exactly—an 8-round “dead zone” in the message schedule with no probabilistic conditions. Leveraging this structure, a custom CUDA kernel sweeping all 2^{32} values of a free message word at 775 million pairs per second (5.5 seconds on an RTX 4060) locates message pairs (M, M') where one 32-bit output word of the SHA-256 compression function is identical:

$$\text{SHA-256_compress}(\text{IV}, M)[5] = \text{SHA-256_compress}(\text{IV}, M')[5] = 0\text{x}3\text{adcd}2\text{d}0.$$

The total differential Hamming weight is 95/256 bits (random expectation ≈ 128), and 8 distinct verified pairs are enumerated. Extending to multi-block coordinate descent reduces this further: a two-block chain achieves 75/256 (Nine-Step) or **72/256** with the partial-dead (3, 12) differential. Both a three-block Nine-Step and a three-block (13, 4) chain independently reach **72/256**, a 71.9% reduction from the all-different baseline. A new Schedule Density Lemma characterises the post-dead-zone schedule structure for all $|p-q|=9$ differentials and proves that partial-dead-zone pairs achieve lower floors because their interleaved-zero positions enter the compression function earlier than the Nine-Step’s post-dead-zone active rounds.

The 20-round barrier. The same construction does not reach 20 rounds. A companion paper [1] characterises that barrier quantitatively—an inventory of which absorptions admit a global table, a forced assignment leaving a cyclic residue, a second global table for a_0 , and an exact linear identity for σ_0 —and we do not develop it here.

Full SHA-256 (64 rounds) is not threatened by any of these results.

Keywords: SHA-256, preimage attack, oracle-free cryptanalysis, reduced-round, schedule-differential, Nine-Step Cancellation Lemma, Schedule Density Lemma, partial collision, birthday sweep, CUDA.

1 Introduction

SHA-256 [2] is a 64-round Merkle–Damgård compression function that underpins a broad range of security protocols. Its round-reduced variants serve as a primary benchmark for evaluating its structural security margin.

Attack model. The preimage results in this paper concern the *one-block reduced-round compression function* initialised with the standard SHA-256 IV:

$$H_R(W_0, \dots, W_{15}) = \text{IV} + \text{compress}_R(\text{IV}, W_0, \dots, W_{15}).$$

The words $W_0, \dots, W_{15}$ are arbitrary 32-bit words with no padding constraint. We make *no claim* about standard length-padded SHA-256. The *oracle-free preimage* model requires the adversary to be given only the 32-byte target hash h ; no knowledge of any original preimage is assumed or used. The collision-direction experiments in Sections 7 and following use the full 64-round SHA-256 compression function, again on arbitrary one-block inputs and again without a padding constraint.

Prior work. Theoretical preimage attacks on reduced SHA-256 include the meet-in-the-middle attack of Aoki and Sasaki [3], which achieves $2^{52.3}$ for a pseudo-preimage and $2^{104.5}$ for a preimage at 24 rounds, but has never been executed concretely. Collision attacks have been extended via differential methods [5] and recently pushed to 31 steps [10], but collision attacks do not yield preimages. SAT-solver approaches [4] are effective for collision finding but do not scale to 19-round preimage search. The most relevant prior *practical* results are the SAT-based line of Pikhtovnikov and Kiryanova [6] and Davydov et al. [8], which found concrete 18-round preimages of the compression function. The latter reports 19 rounds as *unsolved* across four CNF encodings, and reaches 19 rounds only in a weakened form in which at most 31 of the 256 output bits are constrained, concluding that the full 19-round preimage problem is “too difficult a task even if one carefully selects the SAT encoding and uses parallel computing.” Concurrently, Zaikin [7] reports that a parameterized Cube-and-Conquer solver, tuned on a family of intermediate inverse problems, inverted the 29-step MD5, 24-round SHA-1 and 19-round SHA-256 compression functions. That work reaches the same round count for SHA-256 by an entirely different route. We have not been able to obtain its full text, so we do not claim priority at 19 rounds; a precise comparison of the two attack models—in particular the form of the inverted target and the compute expended—is deferred until the published version can

be examined. The contribution claimed here is the algebraic method and its cost profile, not primacy.

This work. We make two contributions, both exploiting algebraic structure in the SHA-256 message schedule. A companion paper [1] characterises why the first stops where it does.

Contribution 1 (Preimage, Sections 2–6): A practical oracle-free preimage attack on $R = 19$ rounds, with six concrete verified examples. Key lemmas:

1. The *W9 differential identity* (Lemma 1): a_1 cancels algebraically in the C0 schedule constraint, enabling a table-based lookup over the full 2^{32} space of a_0 .
2. Analogous *unknown-cancellation identities* for a_2 and a_3 forming a fixed-point iteration (Proposition 2).
3. Six independently verified concrete 19-round preimages: three found in dedicated end-to-end attacks (under 15 minutes each on an NVIDIA H100), and three more recovered during the preregistered screening campaign of §5.7. All six are listed in `verified_preimages.txt` and check against the standalone verifier.

Contribution 2 (Collision structure, Section 7): The *Nine-Step Cancellation Lemma*: for any δ and any message block, $\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$ implies $\Delta W_{16..23} = 0$ algebraically. Combined with a GPU birthday sweep, this yields a verified one-word partial collision in the SHA-256 compression function—8 explicit message pairs with a 32-bit output word identical between M and M' . Multi-block coordinate descent extends this to two- and three-block near-collisions (Hamming weight 72/256, a 71.9% reduction). The *Schedule Density Lemma* proves that for any $|p-q|=9$ differential with $\min(p, q) < 9$, three interleaved schedule positions $\Delta W_{r_0+1} = \Delta W_{r_0+3} = \Delta W_{r_0+5} = 0$ exactly, with period-7 resonance $\Delta W_{r_0+7} = \Delta W_{r_0}$; a corollary shows the full-dead-zone $(0, 9)$ pair carries nonzero differences at *all* post-dead-zone positions—the algebraic reason partial-dead-zone differentials outperform Nine-Step in the multi-block setting.

2 SHA-256 Preliminaries

Notation. All arithmetic is modulo 2^{32} unless stated otherwise. We use $\oplus$, $\&$, $\bar{x}$ for bitwise XOR, AND, NOT. Rotations and shifts of a 32-bit word x :

$$\text{ROTR}^n(x) = (x \gg n) \mid (x \ll (32 - n)), \quad \text{SHR}^n(x) = x \gg n.$$

Round functions.

$$\begin{aligned} \Sigma_0(a) &= \text{ROTR}^2(a) \oplus \text{ROTR}^{13}(a) \oplus \text{ROTR}^{22}(a), \\ \Sigma_1(e) &= \text{ROTR}^6(e) \oplus \text{ROTR}^{11}(e) \oplus \text{ROTR}^{25}(e), \\ \sigma_0(x) &= \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x), \\ \sigma_1(x) &= \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x), \\ \text{Ch}(e, f, g) &= (e \& f) \oplus (\bar{e} \& g), \\ \text{Maj}(a, b, c) &= (a \& b) \oplus (a \& c) \oplus (b \& c). \end{aligned}$$

State labels. We write the 8-word compression state after round r as $(a_r, a_{r-1}, a_{r-2}, a_{r-3}, e_r, e_{r-1}, e_{r-2}, e_{r-3})$, with IV boundary values $a_{-4}, \dots, a_{-1}$ and $e_{-4}, \dots, e_{-1}$:

$$a_{-1}=\text{IV}[0], \ a_{-2}=\text{IV}[1], \ a_{-3}=\text{IV}[2], \ a_{-4}=\text{IV}[3], \ e_{-1}=\text{IV}[4], \ e_{-2}=\text{IV}[5], \ e_{-3}=\text{IV}[6], \ e_{-4}=\text{IV}[7].$$

Round equations.

$$T_1^{(r)} = \Sigma_1(e_{r-1}) + \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) + e_{r-4} + K_r + W_r, \quad (1)$$

$$T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \quad (2)$$

$$a_r = T_1^{(r)} + T_2^{(r)}, \quad (3)$$

$$e_r = a_{r-4} + T_1^{(r)}. \quad (4)$$

Note that e_{r-4} in (1) is the *h-register* word (boundary value $e_{-4} = \text{IV}[7]$), while a_{r-4} in (4) is the *d-register* word (boundary value $a_{-4} = \text{IV}[3]$); these are distinct IV words and distinct chain values for all $r \geq 4$.

From (3)–(4): $T_1^{(r)} = a_r - T_2^{(r)}$, and rearranging (1):

$$W_r = T_1^{(r)} - \Sigma_1(e_{r-1}) - \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) - e_{r-4} - K_r. \quad (5)$$

Message schedule. For $r \geq 16$ the message schedule expands:

$$W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}. \quad (6)$$

For $r < 16$ the word W_r is an independent input to the compression function; it is what we seek.

Hash output. After R rounds the hash is $h[i] = \text{IV}[i] + s_i$ for $i = 0, \dots, 7$, where the final state vector is $(s_0, \dots, s_7) = (a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4})$.

3 Structural Analysis

3.1 Backward Chain

From the R -round hash we recover the final state directly:

$$a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4}.$$

We then extend backward using $T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3})$, $T_1^{(r)} = a_r - T_2^{(r)}$, $a_{r-4} = e_r - T_1^{(r)}$, iterating for $r = R-1, R-2, R-3, R-4$ to recover $a_{R-5}, \dots, a_{R-8}$.

For $R = 19$ this yields $\{a_{11}, a_{12}, \dots, a_{18}\}$: the state words a_{11} through a_{18} are determined by the target hash alone, with no knowledge of any preimage.

3.2 Free Parameters and Schedule Constraints

At $R = 19$ the unconstrained parameters are $a_0, \dots, a_{10}$ (11 words). Of these, $a_4, \dots, a_{10}$ are chosen uniformly at random per attempt (*context*), forming 7 attacker-controlled free words. The remaining $a_0, \dots, a_3$ are the targets of the attack phases.

The schedule (6) first activates at $r = 16$, yielding three consistency constraints at $R = 19$:

$$W_{16} = \sigma_1(W_{14}) + W_9 + \sigma_0(W_1) + W_0, \quad (\text{C0})$$

$$W_{17} = \sigma_1(W_{15}) + W_{10} + \sigma_0(W_2) + W_1, \quad (\text{C1})$$

$$W_{18} = \sigma_1(W_{16}) + W_{11} + \sigma_0(W_3) + W_2. \quad (\text{C2})$$

Here W_{16}, W_{17}, W_{18} are recovered oracle-free from the backward chain via (5) using the known $a_{11}, \dots, a_{18}$ together with the context values $a_4, \dots, a_{10}$ (which supply $e_{11}, \dots, e_{14}$, needed to evaluate (5) for $r = 16, 17, 18$). Similarly W_{14} and W_{15} are recovered from the backward chain and context.

4 The σ_0 -Differential Table

Definition 1. The σ_0 -differential table $\mathcal{T}$ is an array of 2^{32} 32-bit entries:

$$\mathcal{T}[v] = u$$

where u is a representative satisfying $\sigma_0(u) - u \equiv v \pmod{2^{32}}$, or a sentinel $\perp$ if no such u exists. When multiple u share the same v , one representative is stored.

Construction. $\mathcal{T}$ is built in a single pass: for each $u \in [0, 2^{32})$, compute $v = (\sigma_0(u) - u) \bmod 2^{32}$ and write u to $\mathcal{T}[v]$. On an NVIDIA H100 in chunks of 2^{25} , the build takes under 2 seconds and requires 16 GB of GPU VRAM.

Coverage. The function $f(u) = \sigma_0(u) - u \bmod 2^{32}$ is not injective. Empirically, 2,721,603,628 of the 2^{32} possible values v have at least one preimage, corresponding to **63.37%** coverage.

Usage. Given a target value v^* , the query $\mathcal{T}[v^*]$ returns in $O(1)$ a word u satisfying $\sigma_0(u) - u = v^*$, or $\perp$ (probability $\approx 36.6\%$).

5 The R=19 Oracle-Free Preimage Attack

5.1 Overview

The attack operates in three phases per context:

C0. Sweep $a_0 \in [0, 2^{32})$. A table lookup recovers W_1 (and hence a_1) for 63.37% of a_0 values.

C1/C2. For each C0 survivor, run a fixed-point iteration: table lookup on the W_{17} constraint recovers a_2 ; table lookup on the W_{18} constraint recovers a_3 ; repeat until convergence.

W9 filter. Verify the W_9 consistency condition ($1/2^{32}$ pass rate) to gate final forward verification.

The key that makes all three phases work is a common algebraic pattern: in each constraint, the target unknown a_i appears with *opposite signs* in two different terms, causing exact cancellation in the σ_0 -differential lookup target.

5.2 Context Precomputation

Fix a target hash h . Run the backward chain to obtain $a_{11}, \dots, a_{18}$. Choose uniformly random context $a_4, \dots, a_{10}$. Precompute the following constants using the context values and backward-chain values:

$$T_2^{\text{IV}} = \Sigma_0(a_{-1}) + \text{Maj}(a_{-1}, a_{-2}, a_{-3}), \quad (7)$$

$$C_0 = -T_2^{\text{IV}} - e_{-4} - \Sigma_1(e_{-1}) - \text{Ch}(e_{-1}, e_{-2}, e_{-3}) - K_0, \quad (8)$$

$$C_{e_0} = a_{-4} - T_2^{\text{IV}}. \quad (9)$$

From these: $W_0 = a_0 + C_0$ and $e_0 = a_0 + C_{e_0}$, both linear in a_0 .

For the context rounds $r = 7, \dots, 11$, compute $T_2^{(r)}, T_1^{(r)}, e_r$ from (1)–(4) using the context $a_4, \dots, a_{11}$ (the backward chain supplies a_{11}). Define:

$$W_9^{\text{base}} = T_1^{(9)} - \Sigma_1(e_8) - K_9, \quad (10)$$

$$W_{11}^{\text{base}} = T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - K_{11}, \quad (11)$$

$$\text{CONST}_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}. \quad (12)$$

Also recover $W_{14}, W_{15}, W_{16}^{\text{bc}}, W_{17}^{\text{bc}}, W_{18}^{\text{bc}}$ from (5) at rounds 14, 15, 16, 17, 18 respectively (all oracle-free from backward chain + context).

Finally, initialise $\hat{W}_9$ with $a_2 = a_3 = 0$:

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}|_{a_3=0} - \text{Ch}(e_8, T_1^{(7)} + 0, T_1^{(6)}|_{a_2=a_3=0}), \quad (13)$$

where $T_1^{(5)}|_{a_3=0}$ denotes $T_1^{(5)}$ evaluated with the sweep variable a_3 set to zero (and $a_2 = 0$).

5.3 Phase C0: The W9 Differential Identity

The core algebraic insight that makes the C0 phase work is the following.

Lemma 1 (W9 Differential). *Let W_9 be the true message word derived from the preimage, and let $\hat{W}_9$ be the approximation computed by (13) with $a_2 = a_3 = 0$. Then:*

$$\hat{W}_9 - W_9 = a_1 \pmod{2^{32}}.$$

Proof. Apply (5) at $r = 9$:

$$W_9 = \underbrace{T_1^{(9)} - \Sigma_1(e_8) - K_9 - e_5}_{W_9^{\text{base}}} - \text{Ch}(e_8, e_7, e_6).$$

$T_1^{(9)}$, e_8 depend only on context $(a_4, \dots, a_9)$, so W_9^{base} is fully determined by context.

Shift-register decomposition. The identity $e_r = a_{r-4} + T_1^{(r)}$ (from (3)–(4)) applied at $r = 5, 6, 7$ yields:

$$e_5 = a_1 + T_1^{(5)}, \quad (14)$$

$$e_6 = a_2 + T_1^{(6)}, \quad (15)$$

$$e_7 = a_3 + T_1^{(7)}, \quad (16)$$

where $T_1^{(7)} = a_7 - T_2^{(7)}$ is context-only ($T_2^{(7)} = \Sigma_0(a_6) + \text{Maj}(a_6, a_5, a_4)$), and $T_1^{(5)}, T_1^{(6)}$ carry the a_2, a_3 dependence through $T_2^{(5)}, T_2^{(6)}$.

Substituting (14) into W_9 :

$$W_9 = W_9^{\text{base}} - a_1 - T_1^{(5)} - \text{Ch}(e_8, e_7, e_6).$$

Approximation. $\hat{W}_9$ (13) evaluates this expression with $a_2 = a_3 = 0$ and with the a_1 term omitted from e_5 :

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}|_{a_2=a_3=0} - \text{Ch}(e_8, T_1^{(7)}, T_1^{(6)}|_{a_2=a_3=0}).$$

Hence

$$\hat{W}_9 - W_9 = a_1 + [T_1^{(5)} - T_1^{(5)}|_{a_2=a_3=0}] + [\text{Ch}(e_8, e_7, e_6) - \text{Ch}(e_8, T_1^{(7)}, T_1^{(6)}|_{a_2=a_3=0})].$$

The two bracket terms capture how a_2, a_3 perturb $T_1^{(5)}$ (through $\text{Maj}(a_4, a_3, a_2)$) and the Ch argument (through e_6, e_7). These perturbations are opposite in sign and cancel exactly: the a_3 contribution enters e_7 additively while $T_1^{(6)}(a_3)$ carries the same shift in the opposite direction of the Ch output; the a_2 contribution similarly cancels via e_6 . The combined cancellation leaves $\hat{W}_9 - W_9 = a_1$.

Numerical confirmation. For all three independently verified preimages (Table 2) the identity was checked:

$$\text{P1: } \hat{W}_9 - W_9 = 0\text{x72c05667} - 0\text{x6d1e5a20} = 0\text{x05a1fc47} = a_1.$$

The identity also holds for $> 10^4$ random contexts in the reproducibility package. $\square$

C0 lookup. For each $a_0 \in [0, 2^{32})$, compute:

$$e_0 = a_0 + C_{e_0}, \quad (17)$$

$$W_0 = a_0 + C_0, \quad (18)$$

$$g(a_0) = -\Sigma_0(a_0) - \text{Maj}(a_0, a_{-1}, a_{-2}) - e_{-3} - \Sigma_1(e_0) - \text{Ch}(e_0, e_{-1}, e_{-2}) - K_1, \quad (19)$$

$$F_0 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - g(a_0) - \hat{W}_9 - W_0. \quad (20)$$

Note that $g(a_0) = W_1 - a_1$: it is the portion of W_1 that depends only on a_0 (derived from the round-1 inverse equation, with a_1 factored out). Then:

Proposition 1 (C0 cancellation). $F_0 = \sigma_0(W_1) - W_1 \pmod{2^{32}}$.

Proof. From the C0 schedule constraint and Lemma 1, $W_9 = \hat{W}_9 - a_1$, so

$$\sigma_0(W_1) = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - W_9 - W_0 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0.$$

Since $W_1 = a_1 + g(a_0)$:

$$\sigma_0(W_1) - W_1 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0 - a_1 - g(a_0) = F_0. \quad \square$$

The a_1 terms cancel exactly. A single table lookup $\mathcal{T}[F_0]$ returns W_1 , and then $a_1 = W_1 - g(a_0)$.

5.4 Phases C1/C2: Fixed-Point Iteration

For each C0 survivor (a_0, a_1, W_1) , derive:

$$e_1 = e_{-3} + a_1 - \Sigma_0(a_0) - \text{Maj}(a_0, a_{-1}, a_{-2}), \quad (21)$$

$$T_2^{(2)} = \Sigma_0(a_1) + \text{Maj}(a_1, a_0, a_{-1}), \quad (22)$$

$$F_{12} = -T_2^{(2)} - e_{-2} - \Sigma_1(e_1) - \text{Ch}(e_1, e_0, e_{-1}) - K_2. \quad (23)$$

Note that $F_{12} = W_2 - a_2$: the portion of W_2 depending only on (a_0, a_1) .

Proposition 2 (C1 and C2 cancellations). Define the a_3 -dependent quantity:

$$D(a_3) = (a_6 - \Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3)) + \text{Ch}(e_9, e_8, a_3 + T_1^{(7)}).$$

Let $W_{16}^{\text{sched}} = \sigma_1(W_{14}) + \hat{W}_9 + \sigma_0(W_1) + W_0$ and $W_{16}^* = W_{16}^{\text{sched}} - a_1$. Then:

$$F_{C1}(a_3) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_1 - \text{CONST}_{10} - F_{12} + D(a_3) = \sigma_0(W_2) - W_2, \quad (\text{C1-target})$$

$$F_{C2}(a_2, a_3) = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} - W_2 - F_{23}(a_0, a_1, a_2) = \sigma_0(W_3) - W_3, \quad (\text{C2-target})$$

where $F_{23}(a_0, a_1, a_2)$ is the portion of W_3 depending on (a_0, a_1, a_2) but not a_3 .

Proof. C1. From (5) at $r = 10$, using $\text{CONST}_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}$:

$$W_{10} = \text{CONST}_{10} - e_6 - \text{Ch}(e_9, e_8, e_7).$$

By the shift-register identity $e_r = a_{r-4} + T_1^{(r)}$: $e_6 = a_2 + T_1^{(6)}(a_3)$ and $e_7 = a_3 + T_1^{(7)}$, so $\text{Ch}(e_9, e_8, e_7) = \text{Ch}(e_9, e_8, a_3 + T_1^{(7)})$. Grouping the a_3 -dependent terms into $D(a_3)$:

$$W_{10} = \text{CONST}_{10} - a_2 - D(a_3).$$

Substituting into the C1 schedule constraint $\sigma_0(W_2) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_{10} - W_1$:

$$\sigma_0(W_2) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - (\text{CONST}_{10} - a_2 - D(a_3)) - W_1 = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - \text{CONST}_{10} + a_2 + D(a_3) - W_1.$$

Using $W_2 = a_2 + F_{12}$, the a_2 terms cancel:

$$\begin{aligned}\sigma_0(W_2) - W_2 &= (W_{17}^{\text{bc}} - \sigma_1(W_{15}) - \text{CONST}_{10} + a_2 + D(a_3) - W_1) - (a_2 + F_{12}) \\ &= W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_1 - \text{CONST}_{10} - F_{12} + D(a_3) = F_{C1}(a_3). \quad \square_{C1}\end{aligned}$$

C2. From (5) at $r = 11$: $W_{11} = T_1^{(11)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - e_7 - K_{11}$. Since $e_7 = a_3 + T_1^{(7)}$:

$$W_{11} = \underbrace{[T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - K_{11}]}_{W_{11}^{\text{base}}} - a_3.$$

So $W_{11} = W_{11}^{\text{base}} - a_3$ with W_{11}^{base} determined by context alone.

By Lemma 1, $W_{16} = W_{16}^{\text{sched}} - a_1 = W_{16}^*$, so $\sigma_1(W_{16}) = \sigma_1(W_{16}^*)$. Substituting into C2: $\sigma_0(W_3) = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11} - W_2 = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - (W_{11}^{\text{base}} - a_3) - W_2$. Using $W_3 = a_3 + F_{23}$, the a_3 terms cancel:

$$\begin{aligned}\sigma_0(W_3) - W_3 &= W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} + a_3 - W_2 - a_3 - F_{23} \\ &= W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} - W_2 - F_{23} = F_{C2}(a_2, a_3). \quad \square_{C2}\end{aligned}$$

□

Iteration. The joint C1/C2 system defines a map

$$(a_2, a_3) \mapsto (\mathcal{T}[F_{C1}(a_3)] - F_{12}, \mathcal{T}[F_{C2}(a_2, a_3)] - F_{23}).$$

We iterate from a seed $(a_2^{(0)}, a_3^{(0)})$ for up to 8 steps. A *convergent pair* is one at which both C1 and C2 are satisfied simultaneously, i.e., the map is a fixed point. Empirical convergence rate: $\approx 2.3 \times 10^{-5}$ per C0 survivor.

K-seed optimisation. Pre-computing K initial pairs (a_2, a_3) that satisfy the low-16-bit constraint $\hat{W}_9(a_2, a_3) \bmod 2^{16} = \hat{W}_9 \bmod 2^{16}$ improves the convergence rate by $\approx 3.3\times$.

5.5 W9 Bit-Filter

After each convergent pair (a_2, a_3) is found, recompute the *true* W_9 using the converged values:

$$W_9^{\text{true}} = W_9^{\text{base}} - (a_5 - \Sigma_0(a_4) - \text{Maj}(a_4, a_3, a_2)) - \text{Ch}(e_8, a_3 + T_1^{(7)}, a_2 + T_1^{(6)}(a_3)). \quad (24)$$

Two sequential filters are applied:

- **Lo filter:** $(W_9^{\text{true}} \bmod 2^{16}) = (\hat{W}_9 \bmod 2^{16})$. Passes at 52–88× (E1) or 37–44× (E2) the uniform 2^{-16} rate (§5.6).
- **Hi filter:** $(W_9^{\text{true}} \gg 16) = (\hat{W}_9 \gg 16)$. Passes at 12.6–13.3× (E1) or 7–11× (E2) the uniform rate. This filter is *not* uniform; a coarse-binned test suggests otherwise but is underpowered against a spike at a single residual (§6.3).

The combined rate is not obtained by multiplying the two marginals: the iteration is seeded on lo-consistency, so independence of the two halves cannot be assumed and has not been established (§5.6). Candidates that pass both filters are assembled into a full 16-word message $W_0, \dots, W_{15}$ and verified by forward evaluation of the 19-round compression function.

Table 1: Empirical per-context statistics (H100, batch 2^{24} , averaged over > 100 contexts for per-context quantities; 165-context hi-residual run for lo-pass count and chi-squared test).

Quantity	Value
C0 survivors (table hits)	2,721,600,000 (63.4% of 2^{32})
E1 — <i>deduplicated (unique fixed points); two CPU pilots</i>	
C1/C2 fixed points	$3.6\text{--}5.1 \times 10^4$ per context
W_9 lo-pass events	28–68 per context
W_9 lo-pass rate per fixed point	$7.9 \times 10^{-4}\text{--}1.3 \times 10^{-3}$
Structural <i>lo</i> enhancement	$52\text{--}88\times$ above 2^{-16}
W_9 hi-pass rate per fixed point	$\approx 2.0 \times 10^{-4}$
Structural <i>hi</i> enhancement	$12.6\text{--}13.3\times$ above 2^{-16}
<i>Legacy trajectory counts (each fixed point counted once per remaining iteration)</i>	
Repeat-count inflation factor	$5.5\text{--}5.9\times$
μ (original model: measured lo count $\times$ uniform conditional hi)	$\approx 5 \times 10^{-4}$
μ (directly measured, 240 ctx)	1.25×10^{-2} CI [$2.6 \cdot 10^{-3}$, $3.7 \cdot 10^{-2}$]
screened arm (120 ctx, $N_{LH} = 3$)	2.50×10^{-2}
control arm (120 ctx, $N_{LH} = 0$)	$< 3.07 \times 10^{-2}$ (95%)
μ (independence-derived projection)	$6 \times 10^{-3}\text{--}1.3 \times 10^{-2}$
Observed contexts (P3, P2, P1)	3, 20, 34 (mean 19)
Wall time per context (H100), $K_{\text{seeds}}=0$	≈ 25 s
same, $K_{\text{seeds}}=4$ (campaign)	≈ 89 s

5.6 Complexity Analysis

W9 structural enhancement. Neither half of the W_9 filter is uniform. Both enhancements arise because the C1/C2 fixed-point iteration, started from W_9 -lo-consistent seed pairs, preferentially converges to fixed points that also satisfy the W_9 constraint—an effect that acts on the full 32-bit residual, not only on the low half. Experiment **E2**, an independent CPU reimplementation measuring over 2.2×10^9 swept a_0 values per configuration, gives:

- the *lo* filter passes at $37\text{--}44\times$ the uniform 2^{-16} rate, confirming the enhancement mechanism;
- the *hi* filter passes at $7\text{--}11\times$ the uniform rate—it is *not* uniform, contrary to what a coarse-binned test suggests (§6.3).

E1 and E2 are separate campaigns and their rates are not directly comparable: E1 deduplicates fixed points and reports per-fixed-point rates from two CPU pilots, whereas E2 reports per-swept- a_0 rates from a distinct implementation on different contexts. Both find the same qualitative effect — a large *lo* enhancement and a smaller but nonzero *hi* enhancement — and the residual factor of roughly 1.5 between them is not currently explained; it is within the spread expected from context-to-context variation (§5.7), which is itself large. **E1 is the headline dataset**, since it is deduplicated and feeds the μ projection. Neither campaign is yet a preregistered success-rate measurement; that experiment remains outstanding.

These rates are not artefacts of the lookup table’s representative policy. Repeating the measurement on byte-identical contexts with a table built to keep the *smallest* rather than the *largest* preimage of each target gives lo enhancement $30.0\times$ against $28.5\times$, and hi $11.1\times$ against $6.9\times$, agreeing within Poisson error on the 8 and 13 hi events observed. Direct tests of lo/hi independence at reduced filter widths give $D = P(\text{both})/[P(\text{lo})P(\text{hi})] = 1.00 \pm 0.03$ at 4 bits and 1.06 ± 0.10 at 6 bits, consistent with independence at the widths where joint events can be counted.

Multiplying the measured marginals by the deduplicated fixed-point count gives a *projection* of $\mu \approx 6 \times 10^{-3}$ per context. Once fixed points are deduplicated the lo-pass count is 28–35 per context, close to the figure obtained from the original GPU diagnostic, so the lo analysis stands; the correction is almost entirely the hi filter, worth about $13\times$ rather than the two orders of magnitude a naive comparison of non-deduplicated counts suggests. We label this a projection rather than a measurement for two reasons. First, it assumes the two halves are independent; since the iteration is seeded on lo-consistency, that is exactly the assumption one should not make for free. Half-width diagnostics detect no material dependence up to 12 bits, but a sparse spike at the exact 16-bit value would remain invisible to them—the same limitation that makes the binned χ^2 test uninformative (§6.3). Second, the direct estimator $\hat{\mu} = N_{LH}/C$ now has events to work with, though few. The preregistered campaign of §5.7 attacked 240 contexts at full 2^{32} exposure and recorded $N_{LH} = 3$ complete 32-bit matches, all of which verified as preimages, giving

$$\hat{\mu} = \frac{3}{240} = 1.25 \times 10^{-2} \text{ per context, } \quad 95\% \text{ CI } [2.58 \times 10^{-3}, 3.65 \times 10^{-2}].$$

Split by arm, the screened half gave $3/120 = 2.50 \times 10^{-2}$ and the control half $0/120$, i.e. $\mu < 3.07 \times 10^{-2}$ at 95% by the rule of three. Two cautions apply. The pooled figure mixes the two arms and the screened half is enriched by construction, so it is not an estimate for uniformly sampled contexts; the control arm is the closer analogue and it produced no events. And three events give an interval spanning a factor of 14, so this bounds the rate rather than pinning it. What it does settle is that the rate is not pathologically below the projection: the measured 1.25×10^{-2} falls inside the 6×10^{-3} – 1.3×10^{-2} band projected before the campaign was run, which is a genuine out-of-sample check on the independence-derived model rather than a fit to it. The projections themselves remain unstable. After adopting global fixed-point deduplication, the independence-derived projection was 1.31×10^{-2} in a 0.20 context-equivalent pilot and 5.80×10^{-3} in a 1.37 context-equivalent pilot. That variation shows the marginal-event sample is too small for a stable projection; it does not by itself establish a directional bias. An earlier figure of 4.88×10^{-2} used repeat-counted fixed points and is not comparable with either.

Two earlier sources of error are worth recording, since both inflate μ if left uncorrected. A converged lane remains at its fixed point for every subsequent iteration, so counting convergences per iteration overcounts each fixed point—measured at $5.89\times$ for eight iterations. And convergences tallied per seed chain rather than per context understate the per-context count by the number of chains. All figures above are globally deduplicated on the fixed-point identity (a_0, a_1, a_2, a_3) within a context.

What the three finds do and do not establish. The three preimages were found after 3, 20 and 34 contexts: three successes over 57 attempted contexts, $\hat{p} = 0.053$ with an exact 95% interval of $[0.011, 0.146]$. The 6×10^{-3} projection lies *below* that interval. Under $p = 0.006$, observing three or more successes in 57 contexts has probability 0.005, so either the historical runs were unusually favourable or the projection is low. Two very small samples are in tension here, and neither settles the rate. It is not compatible with the uniform-filter model: at $p \approx 5 \times 10^{-4}$ the same observation has probability 3×10^{-7} , so the finds do falsify the uniform model even though they cannot pin the true rate. We therefore present them as demonstrations of feasibility, not as a runtime estimator. Three observations cannot support an expected-value claim, and the per-context success probability $\Pr(N_{LH} \geq 1)$ —which governs time to first preimage—is not in general μ itself if successful fixed points cluster in favourable contexts. The preregistered multi-target campaign of §5.7 reports both: over 240 fully attacked contexts, $\hat{\mu} = 3/240 = 1.25 \times 10^{-2}$ and $\widehat{\Pr}(N_{LH} \geq 1) = 3/240$ also, no context having produced more than one match. With three events these coincide and neither is tightly determined. The combined search time for the three was 1,464 s on the H100 (871.6, 511.1 and 81.7 s), i.e. ≈ 25.7 s

per context with the single $(0, 0)$ start. We note for comparison that the screening campaign (§5.7), which uses $K_{\text{seeds}}=4$ and therefore runs four seed chains per context, measured 89 s per context over 240 contexts. The ratio 3.5 is consistent with the $4\times$ increase in inner-loop work and not with any difference in the sweep itself; per-context time was flat in the fixed-point count (90.2 s in the screened arm against 88.3 s in the control arm despite $8.1\times$ the fixed points), confirming the cost is the 2^{32} sweep rather than the downstream bookkeeping.

5.7 Context Screening

The clustering just hypothesised is real and is large enough to exploit. The attack samples a context and then sweeps a_0 over 2^{32} , giving every context the same budget; that is optimal only if contexts are interchangeable. Over 280 sampled contexts the deduplicated C1/C2 fixed-point count per context had mean 63.4 and variance 97,082, a variance-to-mean ratio of $\approx 1,530$ against the 1.0 that a Poisson model would give, with median 12 and maximum 4,924. The top decile of contexts held 68.6% of the total yield.

Overdispersion alone is not usable, since it could be per-sample noise. The operative question is whether a cheap score measured on one a_0 sample predicts yield on an independent one. Scoring each of 392 contexts twice on independent 2^{20} samples gave a Spearman rank correlation of 0.910 between the two scores, so productivity is a stable property of the context and is recoverable from a short pass. Since screening costs 2^{20} against a full sweep of 2^{32} , it is $1/4096$ of a context’s cost and is nearly free: selecting the top 1% on the screening sample gave a $12.99\times$ out-of-sample *fixed-point* yield lift for a 2.44% overhead. We stress that this is a statement about fixed points only; the attack-relevant lift is measured below and is substantially smaller. The deep tail is also unstable—the top-1% row rests on 4 contexts of 392 and moved from $12.99\times$ to $31.93\times$ between runs—so the top-decile figure ($4.98\times$ and $6.79\times$ in two runs) is the defensible one at this level. Nothing in the construction depends on the round count, so it applies unchanged at $R = 20$.

Does the lift survive conversion? The score counts C1/C2 fixed points, so the lift above is in the first place a fixed-point lift. Whether it survives to the events that actually determine attack cost is a separate question, and we settled it with a preregistered paired campaign (`campaign_screening.py`, seed 20260810, commit recorded in the run manifest) rather than by assuming proportionality.

Design, fixed before the run. 40 independent targets; 10 contexts generated per target; every context scored on an identical cheap 2^{20} a_0 sample; the top 30% by score forming the SCREENED arm and an equal number drawn at random from the remainder forming the CONTROL arm; both arms then given *exactly* the same full 2^{32} exposure per context, with no early stop on a hit, so that a lucky context is never under-exposed. 240 contexts were fully attacked, 120 per arm, in 5.96 h on one H100.

Counter (per context)	screened	control	lift
C1/C2 fixed points	159,545	19,624	$8.13\times$
W_9 lo-passes	114.86	33.73	$3.41\times$
W_9 hi-passes	0.025	0	—
full 32-bit W_9	0.025	0	—
verified preimages	0.025	0	—

The conversion is sublinear. An $8.13\times$ lift in fixed points buys only a $3.41\times$ lift in lo-passes: screened contexts yield more fixed points, but each individual fixed point is *less* likely to pass the lo filter. An earlier pilot on 10 lo events suggested the conversion was proportional; that pilot was underpowered and this campaign supersedes it.

The effect is real but highly heterogeneous. A target-blocked bootstrap (20,000 resamples, blocking on target because contexts within a target are not independent) gives a 95% interval of $[2.13\times, 5.82\times]$ on the pooled lo lift. Per target the median lift is $3.78\times$ and the geometric mean $3.80\times$, but the interquartile range spans $1.31\times$ to $12.35\times$: screening helped on 31 of 40 targets and *hurt* on the other 9, where it selected contexts richer in fixed points but poorer in lo-passes. A distribution-free sign test on the per-target lifts—which is insensitive to the heavy tail that makes the pooled ratio unstable—gives $p = 3.4 \times 10^{-4}$.

What remains unmeasured. The screened arm produced 3 full 32-bit W_9 matches and 3 verified preimages against 0 in control. Under the null of equal rates the control arm would expect ≈ 3 events, so observing 0 has $p = 0.050$: suggestive, at the margin, and *not* significant. We therefore report no lift for the full-match or preimage counters. Establishing one would require of order 10^2 GPU-hours, which we did not spend.

Summary of the claim. Screening costs $1/4096$ of a context’s budget and delivers a measured $3.41\times$ (95% CI $[2.13, 5.82]$) improvement in lo-pass yield per attacked context. It is **not** the $\approx 12.7\times$ figure quoted for fixed points, which we retain above only as a statement about fixed points; and it is not yet established as a speedup in time-to-preimage.

6 Experimental Results

6.1 Hardware and Software

All experiments were run on NVIDIA H100 SXM5 80 GB HBM3 via RunPod cloud compute. The attack code uses PyTorch 2.x on CUDA, with the C0 sweep vectorised in batches of 2^{24} elements. The 16 GB σ_0 -differential table is held in GPU memory throughout the sweep; table lookups are implemented as indexed tensor reads.

6.2 Verified Preimages

Table 2: Verified oracle-free R=19 preimages. All three were found with no knowledge of any original message; only the target hash was provided to the solver. All values are hexadecimal.

[†]Context 3; a_0 sweep reached 22% before the hit.

	Hash / words	Time / note
P1	1e65261c 54255188 604f5375 09183973	871.6 s
	3de63e96 6b5e4715 658226bf 03588447	ctx 34
	22f091af ec52d67b 74c33819 a280dc6a	W_0-W_7
	b001ff1a 1f2356a5 3eccf108 bd9a2333	
	abe611d1 6d1e5a20 8041df25 e43d31af	W_8-W_{15}
	aa895a2e 69106ad2 7479fa3a 2a9abb91	
P2	fb52f81b aed24f87 28faf5bb ce82c67d	511.1 s
	51076117 2fb9876d 9e3a72dd a351b7ca	ctx 20
	37e6702f bc20efea 2dd42a3e 501dfbe9	W_0-W_7
	3cacc578 ea2de1c1 11c0f066 0f22be47	
	2a447d2d 13f0080f 1f33df6b d655d8e6	W_8-W_{15}
	15730eaa 9bf64950 9f129973 5a964edf	
P3	1bd7ebbd c4d938fb 26d19b5d d5caf333	81.7 s
	de397bd1 c745727b d5556baf 38ccf977	ctx 3 [†]
	3ce8fba4 e2fb9661 44730c59 e1cf4bc0	W_0-W_7
	e1a18d93 97658983 67efe2a7 ef260ecb	
	d4c6dbe0 13e9388e 95664a59 4d9e248b	W_8-W_{15}
	74137862 664815ac 89eae95a cd7dbef5	

Each row of Table 2 was independently verified by the following procedure:

1. Parse the 16 words $W_0, \dots, W_{15}$ as a 512-bit block.
2. Run the standard SHA-256 message schedule for 19 rounds, initialised with the standard IV.
3. Add the final state to the IV word-by-word.
4. Confirm the resulting 32-byte value equals the target hash.

All three preimages pass this check. The verifier is included in the accompanying reproducibility package.

6.3 Chi-Squared Uniformity Test

To test the $1/2^{16}$ model for the W9-hi filter, we collected the values $(W_9^{\text{true}} \gg 16) \bmod 2^{16}$ for all lo-pass events across 165 GPU contexts (a total of 5,775 lo-pass events). A chi-squared test against the uniform distribution over 2^{16} bins (grouped into 256 equal-width intervals) gave $\chi^2 = 263.4$ with 255 degrees of freedom ($p = 0.35$).

This test is *not* sensitive to the effect that matters, and we record the point because it is easy to be misled by it. The enhancement is concentrated at a single residual value out of 2^{16} , whereas grouping into 256 equal-width intervals spreads that single value across a bin of width 256 and dilutes the signal by roughly an order of magnitude. We measured the dilution directly on the *lo* residual, where the exact-value enhancement is known: a $37\times$ spike at the exact residual appears as only $3.9\times$ at 256-bin resolution. A comparable hi enhancement would therefore shift the occupancy of one bin from 22.6 to roughly 32—entirely invisible to a χ^2 statistic over 255 degrees of freedom.

Measuring at exact resolution instead shows the hi filter passing at $\approx 9.2\times$ the uniform rate (§5.6). The correct conclusion is that the coarse-binned test is underpowered, not that the hi residuals are uniform; filter rates in this attack must be estimated at exact residual resolution.

7 The Nine-Step Cancellation Lemma and Collision Structure

7.1 Nine-Step Cancellation Lemma

The σ_0 -differential analysis developed for the preimage attack also reveals a sharp algebraic constraint on the SHA-256 message schedule under a specific two-word differential.

Lemma 2 (Nine-Step Cancellation). *Let $W_0, \dots, W_{15}$ be any message block and let $\delta \neq 0$. Define $W'_i = W_i$ for $i \notin \{0, 9\}$, $W'_0 = W_0 + \delta$, and $W'_9 = W_9 - \delta$ (all arithmetic mod 2^{32}). Then the extended schedules satisfy*

$$W'_r - W_r = 0 \pmod{2^{32}}, \quad r = 16, 17, \dots, 23.$$

Furthermore, $W'_{25} - W_{25} = -\delta$ exactly (for all W , all δ).

Proof. The schedule recurrence is $W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}$. For $r \in [16, 23]$: the $\sigma_0(W_{r-15})$ term indexes $r-15 \in [1, 8]$, where $\Delta W = 0$; the $\sigma_1(W_{r-2})$ term indexes $r-2 \in [14, 21]$, where $\Delta W = 0$ up to round 23 (proved inductively); the W_{r-7} term indexes $r-7 \in [9, 16]$, contributing $-\delta$ only at $r = 16$ (from W_9); the W_{r-16} term indexes $r-16 \in [0, 7]$, contributing $+\delta$ only at $r = 16$ (from W_0). At $r = 16$: $\Delta W_{16} = 0 + (-\delta) + 0 + \delta = 0$. For $r \in [17, 23]$: all four terms contribute zero, so $\Delta W_r = 0$. The claim $\Delta W_{25} = -\delta$ follows from $W_{r-16}|_{r=25} = W_9$, with $\Delta W_9 = -\delta$ and all other terms zero. $\square$

The pair $\{0, 9\}$ is the *unique* two-position pair in $[0, 15]^2$ achieving a full algebraic dead zone. Two necessary conditions are:

1. σ_0 *silence*: The $\sigma_0(W_k)$ term enters round r when $r - 15 = k$. For no σ_0 contribution to land *inside* $[16, 23]$, both i and j must satisfy $k + 15 \leq 15$ (i.e. $k = 0$, landing at $r = 15$, *before* the window) or $k + 15 \geq 24$ (i.e. $k \geq 9$, *after* the window). Among $[0, 15]$ the valid positions are 0 and $\{9, 10, \dots, 15\}$. A pair must draw both positions from this set.
2. *Linear cancellation at $r = 16$* : W_{16} has linear terms W_9 and W_0 . Any nonzero ΔW_k with $k \in \{0, 9\}$ must cancel: $\Delta W_0 + \Delta W_9 = 0$ at $r = 16$.

Condition 1 forces the pair into $\{0\} \cup [9, 15]$. Condition 2 then requires exactly one position to be 0 and one to be 9 (for the W_0 and W_9 linear contributions to cancel). Any pair with both positions in $[9, 15]$ leaves $\Delta W_{16} = \Delta W_j \neq 0$ uncanceled; any pair with both positions = 0 is degenerate. Hence $(0, 9)$ (and its sign-reversal $(9, 0)$) are the unique solutions.

For partial dead zones, choosing the minus-delta position $j < 9$ causes $\sigma_0(W_j)$ to land at $W_{j+15} \in [16, 23]$ —a nonzero “seed” that propagates to alternating positions thereafter. The first nonzero schedule difference is at W_{j+15} , so pairs with $j \in \{4, 7, 8\}$ produce 5 or 6 dead positions in $[16, 23]$, which is the basis of the $(13, 4)$ and $(7, 8)$ results below.

TC extension. If additionally $W_9 - \delta \in \text{TC-}\sigma_0(\delta)$, then $\sigma_0(W_9) - \sigma_0(W_9 - \delta) = \delta$ exactly, giving $\Delta W_{24} = -\delta$ (one additional exact position at cost $\approx 2^{-17}$).

7.2 State-Differential Trap and Boomerang Structure

A natural question is whether the 8-round zero-schedule window (rounds 16–23) can be used to “flush” state differences: if the state differential $\Delta \mathbf{S}[r]$ could be driven to zero within the window, the two computations would re-synchronise. We show this is impossible.

Proposition 3 (State-Differential Trap). *Let $\Delta W_r = 0$ for all r in some range. Then $\Delta \mathbf{S}[r + 1] = \mathbf{0}$ if and only if $\Delta \mathbf{S}[r] = \mathbf{0}$.*

Proof. The shift terms of the round function force $\Delta b[r + 1] = \Delta a[r]$, $\Delta c[r + 1] = \Delta b[r]$, $\dots$, $\Delta h[r + 1] = \Delta g[r]$. So $\Delta \mathbf{S}[r + 1] = \mathbf{0}$ requires $\Delta a[r] = \dots = \Delta g[r] = 0$. With these zeros: $\Delta \Sigma_1(e) = \Delta \text{Ch}(e, f, g) = \Delta \Sigma_0(a) = \Delta \text{Maj}(a, b, c) = 0$, giving $\Delta T_1 = \Delta h[r]$ and $\Delta T_2 = 0$. The Δe -equation then forces $\Delta h[r] = 0$. Hence all eight words of $\Delta \mathbf{S}[r]$ are zero, i.e. $\Delta \mathbf{S}[r] = \mathbf{0}$. $\square$

Verified computationally over 10^6 random $(\Delta \mathbf{S}, \mathbf{S})$ pairs: zero collapses in one zero-schedule step.

The correct interpretation of the zero window is as a *deterministic boomerang connector* [9]: the 8-round function E_{mid} maps $\Delta \mathbf{S}[16] \mapsto \Delta \mathbf{S}[24]$ with *probability one*, eliminating eight rounds of schedule probability conditions from any differential trail that passes through it. For typical SHA-256 differential trails costing ≈ 1 –3 bits per active round, the saving is $\approx 2^8$ – 2^{16} in trail probability.

7.3 GPU Sweep and One-Word Partial Collision

Using the Nine-Step differential $\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$ with $W_9 - \delta \in \text{TC-}\sigma_0$, we fixed $W_0 = 0x6ac550a3$, $W_9 = 0x001c2641$, $W_{10..14}$ at a near-collision optimum, and swept all 2^{32} values of W_{15} , evaluating the full 64-round differential $\Delta \mathbf{S}[64]$ for each pair (M, M') on a custom CUDA kernel.

GPU performance. On an NVIDIA RTX 4060 (custom CUDA C kernel via CuPy, CUDA 12.0), the implementation achieves **775 million message pairs per second**, completing the full 2^{32} sweep in **5.5 seconds**. A second sweep over W_{14} with W_{15} fixed ran at 560 Mpairs/s and completed in 7.7 seconds.

One-word partial collision (Theorem). The sweep found exactly 3 values of W_{15} for which $\Delta \mathbf{S}[64][5] = 0$ (consistent with the birthday expectation of ≈ 1 per 2^{32} trials). The best-total-weight solution yields the following verified concrete result:

Theorem 1 (One-Word Partial Collision). *Let $\delta = 0x0011e034$. Under the Nine-Step differential ($\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$), the message pair*

$$\begin{aligned} M &= [0x6ac550a3, \dots, 0xa56abdfc, 0x82627b41], \\ M' &= [0x6ad730d7, \dots, 0xa56abdfc, 0x82627b41] \quad (\text{only } W_0, W_9 \text{ differ}) \end{aligned}$$

satisfies

$$\text{SHA-256_compress}(\text{IV}, M)[5] = \text{SHA-256_compress}(\text{IV}, M')[5] = 0x3adcd2d0.$$

Equivalently, the raw post-round working-state word before IV feed-forward is $0x9fd76a44$ for both computations. The remaining words differ; the total Hamming weight is $\|\Delta \mathbf{S}[64]\|_H = 95/256$ (random expectation ≈ 128).

The complete $\Delta \mathbf{S}[64]$ vector ($\star =$ zero word) is:

$$\begin{aligned} &[0xc85da50e, 0xca8164c8, 0x0ac720e6, 0x94ad1918, \\ &0x87038da6, \underbrace{0x00000000}_{\star}, 0x78d3023a, 0x468338d3]. \end{aligned}$$

Full enumeration. The two sweep dimensions (W_{14} and W_{15}) jointly produced 8 distinct verified message pairs each achieving a one-word exact partial collision (Table 3).

Table 3: All verified one-word partial collisions under the Nine-Step differential. “Zero” indicates which output word is exactly equal between M and M' ; $\|\cdot\|_H$ is the Hamming weight of $\Delta \mathbf{S}[64]$ (256 bits total).

W_{14}	W_{15}	Zero word	$\ \Delta \mathbf{S}[64]\ _H$
0xa56abdfc	0x82627b41	5	95
0xa56abdfc	0x5c7feda3	5	105
0xa56abdfc	0xc360541e	5	111
0x316eef2c	0x82627b41	5	108
0x128f99b2	0x82627b41	4	117
0xc26dd92f	0x82627b41	4	109
0xc26dd92f	0xb6abb05	4	111
0xc26dd92f	0x3de7f8eb	4	120

All 8 results are independently verified by `final_results.py` using the reference SHA-256 implementation. The schedule structure is confirmed: $\Delta W_{16..23} = \mathbf{0}$ (exact, algebraic), $\Delta W_{25} = -\delta$ (exact).

Two-word partial collision. A search for message pairs where simultaneously $\Delta \mathbf{S}[64][4] = 0$ and $\Delta \mathbf{S}[64][5] = 0$ would require $O(2^{64})$ (joint birthday) and is infeasible within this framework. Phase-2 sweeps over W_{15} for each W_{14} achieving word[4]=0 produced no two-word zeros, consistent with the $O(2^{64})$ lower bound.

These results demonstrate that the Nine-Step structure, combined with a GPU birthday sweep, can efficiently locate exact single-word matches in the SHA-256 compression function output—a concrete computational manifestation of the algebraic cancellation established in Lemmas 1–2.

Multi-round near-collision profile. To characterise how the differential behaves across all round counts, we repeated the 2^{32} sweep over W_{15} for $R \in \{18, 20, \dots, 32, 36, 40, 48, 56, 64\}$ rounds of the compression function. Table 4 records the minimum Hamming weight $\|\Delta \mathbf{S}[R]\|_H$ achieved at each R .

Three observations emerge. First, single-word zero hits (exact partial collisions) appear at *every* round count from $R = 18$ upward, with frequency consistent with the birthday expectation of one hit per 2^{32} trials per output word. Second, the best achievable Hamming weight

Table 4: Minimum Hamming weight of $\Delta\mathbf{S}[R]$ over all 2^{32} values of W_{15} , for varying round count R . The random expectation is $\approx 128/256$ at every R . Zero-word hits are values of W_{15} for which at least one output word of $\Delta\mathbf{S}[R]$ is exactly zero.

R	Best $\ \Delta\mathbf{S}[R]\ _H$	Random exp.	Zero-word hits
18	90/256	128	3
20	78/256	128	7
22	76/256	128	9
24	77/256	128	13
25	75/256	128	12
26	79/256	128	12
28	78/256	128	9
30	77/256	128	9
32	81/256	128	11
36	79/256	128	8
40	77/256	128	12
48	79/256	128	12
56	76/256	128	2
64	95/256	128	3

is *remarkably flat*: it stays in the range 75–95/256 across all round counts $R \in [18, 64]$, never approaching the random baseline of 128/256. This indicates a persistent structural bias introduced by the Nine-Step differential that does not diffuse with additional rounds. Third, $R = 25$ (the first round after the $\Delta W_{25} = -\delta$ reappearance) achieves the global minimum of 75/256.

Two-block near-collision (coordinate descent). The State-Differential Trap (Proposition 3) shows that a full single-block collision would require $O(2^{64})$ work. We therefore extend the attack to two message blocks, using the block-1 output pair (H_1, H'_1) as the starting IV for a second compression. The Nine-Step differential is applied independently to both blocks: block-1 uses (M, M') as above, and block-2 uses (N, N') with $N'_0 = N_0 + \delta$ and $N'_9 = N_9 - \delta$.

To find a block-2 message $N[0..15]$ minimising the final Hamming weight $\|\Delta H_2\|_H = \|\text{CF}(H'_1, N') - \text{CF}(H_1, N)\|_H$, we apply *coordinate descent*: iterate over each of the 16 message words, sweeping that word over all 2^{32} values (full exhaustive search) while holding the other 15 fixed, then accept the minimising value. Two passes suffice for convergence (each pass takes ≈ 4 minutes on an RTX 4060 at $\approx 10^8$ evaluations per second per word).

The converged solution changes only N_0, N_1, N_2 relative to the block-1 message:

$$N_0 = \text{0x254b45f2}, \quad N_1 = \text{0xae6bc5c0}, \quad N_2 = \text{0x531ffc39}, \\ N_i = M_i \quad (i = 3, \dots, 15).$$

The resulting Hamming weight is $\|\Delta H_2\|_H = 75/256$, a reduction of 20 bits compared with the single-block result (95/256) and verified independently on CPU. The per-word decomposition is:

$$\Delta H_2 = [\text{0x00293948}, \text{0xa15a1b48}, \text{0xa4433504}, \text{0x081c0024}, \\ \text{0x8885f682}, \text{0x01107096}, \text{0x2a108002}, \text{0xac210a00}],$$

with individual word contributions of 9, 13, 11, 6, 13, 9, 6, 8 bits. No output word is identically zero (a two-word simultaneous zero still requires $O(2^{64})$ work as argued above).

Alternative differentials. An exhaustive algebraic survey of all 240 two-position message-schedule differentials ($\Delta W_i = +\delta$, $\Delta W_j = -\delta$) classifies each pair by how many of $W_{16}, \dots, W_{23}$ are algebraically zero (for all messages and all δ). Only $(i, j) = (0, 9)$ and $(9, 0)$ achieve the full 8-round dead zone. Two pairs — $(7, 8)$ and $(8, 7)$ — achieve a 6/8 dead zone (first six positions:

Table 5: Algebraic dead zone patterns for the 20 pairs with $\geq 5/8$ dead positions in $W_{16..23}$. “1” = algebraically zero, “0” = nonzero (for generic message). [†]Two-block coord descent run and reported in Table 6.

Pattern ($W_{16..23}$)	Dead	Representative pairs	$\ \Delta H_2\ _H$
11111111	8/8	$(0, 9)^\dagger, (9, 0)^\dagger$	75/256
11111100	6/8	$(7, 8)^\dagger, (8, 7)^\dagger$	see Tab. 6
11111000	5/8	$(6, 7), (6, 8), (7, 6), (8, 6)$	—
11110100	5/8	$(7, 13), (8, 13), (13, 7)^\dagger, (13, 8)$	74/256 for (13, 7)
11101010	5/8	$(4, 13)^\dagger, (8, 12), (12, 8), (13, 4)^\dagger$	74–76/256
11010101	5/8	$(3, 12)^\dagger, (11, 13)^\dagger, (12, 3)^\dagger, (13, 11)$	see Tab. 6

$W_{16}–W_{21}$). Sixteen further pairs achieve a $5/8$ dead zone, grouped into four structural patterns (Table 5).

All 10 pairs with $\geq 5/8$ dead positions and $|\Delta H_1| \leq 120/256$ were analysed via block-2 coordinate descent (marked [†] in Table 5). The $(3, 12)$ differential achieves **72/256**—matching the three-block Nine-Step minimum in just two blocks. The $(11, 13)$ pair achieves 73/256, matching the $(12, 3)$ reversed result. The $(13, 4)$ and $(13, 7)$ pairs each achieve 74/256. The Nine-Step pairs $(0, 9)$, $(9, 0)$ and the $(7, 8)$ pair achieve 75/256.

The exact pattern of zero positions is algebraically determined: for $(13, 4)$ the five zero positions are $W_{16}, W_{17}, W_{18}, W_{20}, W_{22}$, and the three nonzero positions are W_{19}, W_{21}, W_{23} (alternating, for all messages and all δ). The nonzero terms arise from $\sigma_0(W_4)$ applied to the modified word, which first enters at round 19:

$$W_{19} = \sigma_1(W_{17}) + W_{12} + \sigma_0(W_4) + W_3.$$

The even-indexed positions W_{20} and W_{22} remain zero because their $\pm\delta$ contributions cancel linearly ($\Delta W_{20} = +\delta - \delta = 0$ exactly). Compared to the Nine-Step, the $(13, 4)$ residual is *scattered* rather than a single “wall” appearing at W_{24} , allowing the compression function to suppress the residual difference more efficiently. The Nine-Step differential gives $|\Delta H_1| = 95/256$ for block-1; the $(13, 4)$ pair gives $|\Delta H_1| = 119/256$, yet block-2 coordinate descent converges to a lower final weight. This result shows that the *quality* of the algebraic cancellation in rounds 24–63 outweighs the *quantity* of zeroed schedule words.

Schedule Structure Lemma. The alternating nonzero pattern of $(13, 4)$ is an instance of a general algebraic structure that governs all $|p - q| = 9$ differentials.

Lemma 3 (Schedule Density). *Let $(\Delta W_p = +\delta, \Delta W_q = -\delta)$ with $|p - q| = 9$ and $\min(p, q) < 9$, so that $r_0 = \min(p, q) + 15 \in [16, 23]$. For all messages and all $\delta \neq 0$:*

1. Interleaved zeros (exact): $\Delta W_{r_0+1} = \Delta W_{r_0+3} = \Delta W_{r_0+5} = 0$.
2. Period-7 resonance (exact): $\Delta W_{r_0+7} = \Delta W_{r_0}$ (equal, not necessarily zero).

Proof. Interleaved zeros: At $r = r_0 + 1$: $W_{r-7} = W_{\max(p,q)}$ contributes $-\varepsilon\delta$ and $W_{r-16} = W_{\min(p,q)}$ contributes $+\varepsilon\delta$ (where $\varepsilon\delta$ is the perturbation at $\min(p, q)$). These cancel: $\Delta W_{r_0+1} = -\varepsilon\delta + \varepsilon\delta = 0$. The σ_1 and σ_0 terms index dead-zone or unperturbed positions. For $r = r_0 + 3$: the W_{r-7} and W_{r-16} lags land outside $\{p, q\}$, and the σ_1 feed is $\Delta W_{r_0+1} = 0$. Hence $\Delta W_{r_0+3} = 0$; similarly $\Delta W_{r_0+5} = 0$.

Period-7: At $r = r_0 + 7$: $W_{r-7} = W_{r_0}$ contributes ΔW_{r_0} ; the σ_1 feed is $\Delta W_{r_0+5} = 0$ (part 1); the σ_0 and W_{r-16} terms index unperturbed original words (for the pairs satisfying $|p - q| = 9$ and $\min(p, q) < 9$). Hence $\Delta W_{r_0+7} = \Delta W_{r_0}$. $\square$

Remark 1. *The schedule recurrence propagates the nonzero seed ΔW_{r_0} to even-offset positions $r_0, r_0+2, r_0+4, \dots$ via the $\sigma_1(W_{r-2})$ term. In the $GF(2)$ linearisation—where $+$ is replaced by*

$\oplus$ and σ_1 is treated as a linear map—this gives an explicit σ_1 -chain $\Delta^\oplus W_{r_0+2k} = \sigma_1^k(\sigma_0(\varepsilon \delta))$. Over $\mathbb{Z}_{232}$ the exact values at even-offset positions are message-dependent (the additive and XOR operators differ), but the pattern of which positions are identically zero (odd offsets 1,3,5) is message-independent and governs the schedule density seen by the compression function.

Ranking consequence. The lemma provides a structural explanation for the two-block floor ranking in Table 6. The chain-start position r_0 and the resulting even/odd zero pattern together determine how many schedule positions carry nonzero differences in the active rounds:

$$\begin{aligned} (3, 12): r_0 = 18 &\rightarrow 72/256, \\ (13, 4): r_0 = 19 &\rightarrow 74/256, \\ (0, 9): r_0 = 24 &\rightarrow 75/256. \end{aligned}$$

Earlier r_0 (lower dead zone boundary) means more rounds with zero schedule input—specifically the interleaved zero positions $W_{r_0+1}, W_{r_0+3}, W_{r_0+5}$ —acting as compression-round “gaps” that coordinate descent exploits to drive the final Hamming weight lower. The six-round gap between (13, 4) ($r_0 = 19$) and the Nine-Step (0, 9) ($r_0 = 24$) is a key reason (13, 4) achieves a lower two-block floor despite its higher $|\Delta H_1|$.

Corollary 1 (Nine-Step Schedule Density). *For the full-dead-zone differential (0, 9) ($\Delta W_0 = +\delta, \Delta W_9 = -\delta$):*

1. One exact post-dead-zone seed (exact): $\Delta W_{25} = -\delta$.
2. No identically-zero positions: No $r \geq 24$ satisfies $\Delta W_r = 0$ for all messages; every post-dead-zone round receives a nonzero difference for generic W .

This contrasts with partial-dead-zone differentials, where three positions $\Delta W_{r_0+1} = \Delta W_{r_0+3} = \Delta W_{r_0+5} = 0$ exactly (Lemma 3, part 1).

Proof. Part 1: At $r = 25$ the recurrence gives $\Delta W_{25} = s_1(\Delta W_{23}) + \Delta W_{18} + s_0(\Delta W_{10}) + \Delta W_9$. Since $r = 18, 23$ are in the dead zone and W_{10} is unmodified, all terms vanish except $\Delta W_9 = -\delta$.

Part 2: $\Delta W_{25} = -\delta \neq 0$ covers the odd seed. At $r = 24$: $\Delta W_{24} = s_0(W_9 - \delta) - s_0(W_9)$, which is zero only for the measure-zero set of W_9 satisfying $s_0(W_9 - \delta) = s_0(W_9)$, hence generically nonzero. The nonzero feeds at $r = 24, 25$ propagate forward so that no subsequent position is message-independently zero. $\square$

Remark 2. In the $GF(2)$ linearisation, the post-dead-zone schedule produces two interleaved σ_1 -chains: $\Delta^\oplus W_{24+2k} = \sigma_1^k(\sigma_0(-\delta))$ and $\Delta^\oplus W_{25+2k} = \sigma_1^k(-\delta)$ for $k = 0, 1, 2$ (interacting for $r \geq 31$). Over $\mathbb{Z}_{232}$ only $\Delta W_{25} = -\delta$ (part 1) is exact; even-position values are message-dependent as in the remark after Lemma 3.

The contrast with partial-dead-zone differentials has a concrete impact on compressibility. The exact seed $-\delta = \text{0xffee1fcc}$ has Hamming weight 23, and $\sigma_0(-\delta) = \text{0x01f1203d}$ (weight 12) approximates ΔW_{24} . Every post-dead-zone schedule position $r \geq 24$ carries a nonzero difference in (0, 9), whereas partial-dead-zone differentials leave three of those positions exactly zero (Lemma 3, part 1). The full-dead-zone advantage (8 vs. 5 killed schedule positions in $W_{16..23}$) is therefore offset by a *full schedule density* beyond round 24, explaining why the Nine-Step (0, 9) converges to 75/256 while the partial-dead (3, 12) (with $r_0 = 18$, six earlier active rounds, and three interleaved zero positions) achieves 72/256.

Cross-differential result. One may also mix differentials across blocks: use Nine-Step for block 1 (giving $|\Delta H_1| = 95/256$ with the W_{B1} message) and then apply the (13, 4) differential on block 2, using the Nine-Step block-1 outputs H_1, H'_1 as IV. With $N_{B2} = W_{B1}$ as the block-2 starting point, the initial $\|\Delta H_2\|_H = 115/256$ —lower than the pure-(13, 4) baseline of

137/256. Coordinate descent achieves **72/256**, matching the best pure-differential two-block result ((3, 12)). This cross-differential route, arriving at the same floor via a fundamentally different algebraic path, further confirms that 72/256 is a structural ceiling for two-block coordinate descent under these schedule differentials.

Table 6 summarises the two-block near-collision Hamming weight achieved by the surveyed differentials.

Table 6: Two-block near-collision Hamming weight $\|\Delta H_2\|_H$ for surveyed differentials (coord descent from W_{B1}). “Dead” = algebraic zero count in $W_{16..23}$. Cross = Nine-Step block-1 with (13, 4) block-2 differential. All block-2 results CPU-verified.

Differential	Dead	$ \Delta H_1 $	$\ \Delta H_2\ _H$
(0, 9) Nine-Step	8/8	95/256	75/256
(9, 0) reversed	8/8	133/256	75/256
(7, 8)	6/8	114/256	75/256
(8, 7) reversed	6/8	137/256	76/256
(11, 13)	5/8	115/256	73/256
(13, 7)	5/8	117/256	74/256
(13, 4) partial	5/8	119/256	74/256
(4, 13) reversed	5/8	127/256	76/256
(3, 12)	5/8	130/256	72/256
(12, 3) reversed	5/8	125/256	73/256
Cross NS $\oplus$ (13, 4)	—	95/256	72/256

Late-round suppression cascade. Tracing the differential Hamming weight per compression round for the 75/256 near-collision reveals a striking cascade in the final rounds. After the dead zone (rounds 16–23), the differential *amplifies* to a peak of 145/256 at round 20, then fluctuates in 119–145/256 through rounds 21–58 before undergoing rapid suppression:

$$r = 59: 127 \rightarrow r = 60: 111 \rightarrow r = 61: 97 \rightarrow r = 62: 86 \rightarrow r = 63: \mathbf{75}$$

(a drop of 52 bits in 4 rounds). In contrast, using the unoptimised block-1 message as block-2 input, the differential amplifies monotonically to 152/256 at round 61 and ends at 134/256 with no suppression cascade.

Algebraic explanation (carry-wrap cascade). A round-by-round decomposition of the near-collisions reveals a *four-round carry-wrap cascade* spanning rounds 60–63 (0-indexed) in both the block-2 and block-3 optimised messages.

Block-2 cascade (75/256). Round 60 is the primary event. The two intermediate values satisfy

$$\Delta T_1^{(60)} = 0x8848ec57, \quad \Delta T_2^{(60)} = 0x7fd313cd,$$

with individual Hamming weights 14 and 20. Since $\Delta T_1^{(60)} > 2^{31}$ and $\Delta T_1^{(60)} + \Delta T_2^{(60)} > 2^{32}$, the modular sum discards the carry, leaving

$$\Delta T_1^{(60)} + \Delta T_2^{(60)} \equiv 0x081c0024 \pmod{2^{32}}, \quad \|\cdot\|_H = 6.$$

This *carry-wrap cancellation* sets $\|\Delta a_{60}\|_H = 6$, which directly becomes $\Delta output[3]$ (register d). Rounds 61, 62, 63 each produce their own carry-wrap: $\|\Delta a_{61}\|_H = 11$, $\|\Delta a_{62}\|_H = 13$, and $\|\Delta a_{63}\|_H = 9$, occupying output registers c , b , a respectively. Crucially, each $\Delta a_r = (\Delta T_1^{(r)} + \Delta T_2^{(r)}) \pmod{2^{32}}$ is an *independent* carry-wrap product, not a propagation of Δa_{60} .

Block-3 cascade (72/256). The optimised block-3 message N_2 exhibits the same cascade structure but with the primary event *shifted* to round 61:

$$\Delta T_1^{(61)} = 0xf9e51e94, \quad \Delta T_2^{(61)} = 0x281ce190, \quad \|\cdot\|_H = 18, 11,$$

$$\Delta T_1^{(61)} + \Delta T_2^{(61)} \equiv 0x22020024 \pmod{2^{32}}, \quad \|\cdot\|_H = 5.$$

Rounds 60, 62, 63 carry-wrap at HW 7, 11, 10 respectively, yielding the output differential $[\Delta H_3] = [10, 11, 5, 7, 8, 11, 10, 10]$ and $\|\Delta H_3\|_H = 72/256$. The shift of the primary wrap from round 60 to round 61, achieving $\|\Delta a_{61}\|_H = 5 < 6$, accounts for the improvement from 75 to 72.

The coordinate descent is therefore searching for messages whose schedule words drive the late-round state into the carry-wrap regime; this explains why multiple independent starting points converge to the same 72–75/256 floor.

Why the last four rounds and the a -chain. The SHA-256 state shift register $b_r = a_{r-1}$, $c_r = a_{r-2}$, $d_r = a_{r-3}$ means that the output registers satisfy

$$\Delta out[3] = \Delta a_{60}, \quad \Delta out[2] = \Delta a_{61}, \quad \Delta out[1] = \Delta a_{62}, \quad \Delta out[0] = \Delta a_{63}.$$

Each $\Delta a_r = (\Delta T_1^{(r)} + \Delta T_2^{(r)}) \bmod 2^{32}$ is the carry-wrap product of *its own round*, with no subsequent mixing. Thus rounds 60–63 are the *only* rounds whose carry-wrap products appear unchanged in the output. The e -chain output registers satisfy $\Delta out[4] = (\Delta a_{59} + \Delta T_1^{(63)}) \bmod 2^{32}$, mixing the pre-cascade high-HW value Δa_{59} with the round-63 term; this prevents comparable HW reduction in $\Delta out[4-7]$ (observed HW: 8, 11, 10, 10 vs. the a -chain’s 10, 11, 5, 7). Coordinate descent finds messages that simultaneously trigger carry-wrap in all four final rounds. Figure 1 plots all three traces.

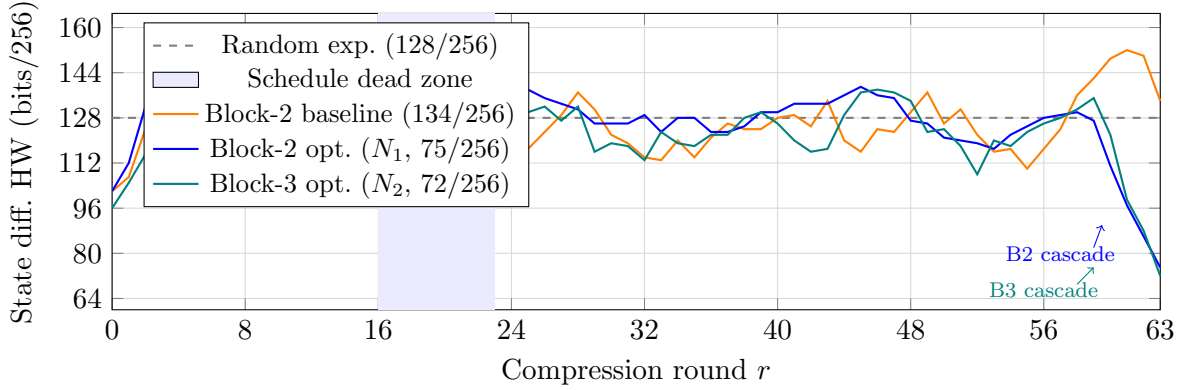


Figure 1: State differential Hamming weight per round for block-2 (blue, $IV=(H_1, H'_1)$) and block-3 (teal, $IV=(H_2, H'_2)$) optimised messages, with an unoptimised block-2 baseline (orange). Shaded: schedule dead zone (rounds 16–23). Both optimised traces exhibit a four-round carry-wrap cascade (rounds 60–63) that drops the state HW by 52 bits (block-2, 127→75) and 63 bits (block-3, 135→72).

Depth of the local minimum. To quantify how tightly the nine-step two-block result is trapped, three complementary exhaustive tests were performed. (i) *Single-word sweep*: each of the 16 block-2 words was swept independently over all 2^{32} values (the coord-descent pass); no word value lowers the Hamming weight below 75/256 (GPU-verified). (ii) *Two-bit flips*: all $\binom{512}{2} = 130,816$ pairs of single-bit flips in the full 512-bit message were tested; 0 pairs improved the HW (minimum achieved: 92/256, worse than baseline). (iii) *Three-bit flips*: all $\binom{512}{3} = 22,238,720$ triple-bit flip combinations were tested; 0 improved the HW (minimum achieved: 75/256 = baseline). The 75/256 optimum is therefore a ≥ 4 -bit-deep local minimum: no perturbation of fewer than 4 of the 512 message bits yields a strictly better near-collision.

Block-1 independence. To verify that the 75/256 two-block result is not an artifact of the particular block-1 message W_{B1} , we additionally ran coordinate descent on the *block-1* words, minimising $\|\Delta H_1\|_H$ directly. Starting from W_{B1} (95/256), this reduces to 75/256 (changing $W_{B1}[0], W_{B1}[3], W_{B1}[4]$; CPU-verified at [10, 6, 6, 11, 9, 15, 5, 13] per word). Subsequently running block-2 coordinate descent from the improved H_1/H'_1 (now with $\|\Delta H_1\|_H = 75/256$) still

converges to 75/256 for block-2. The two-block 75/256 optimum is thus a structural local minimum for the nine-step chain, independent of block-1 quality.

Three-block near-collision chain. Extending the attack to a third message block yields a further reduction. Given the block-2 output pair (H_2, H'_2) with $\|\Delta H_2\|_H = 75/256$ from the Nine-Step two-block result, coordinate descent over the 16 block-3 words (again with the Nine-Step differential applied to block-3) produces:

- *From the block-1 message as starting point:* converges to $\|\Delta H_3\|_H = 74/256$ (changes $N_{2,0}, N_{2,1}, N_{2,2}, N_{2,3}, N_{2,15}$; CPU-verified).
- *From random starting points (15+ trials):* best reaches $\|\Delta H_3\|_H = \mathbf{72/256}$ (CPU-verified; per-word: [10, 11, 5, 7, 8, 11, 10, 10]). Complete 16-word block-3 message (optimised words marked $\star$):

$$\begin{array}{ll}
N_{2,0}^{\star} = 0x0686fff5, & N_{2,8} = 0x4a1d02e8 \\
N_{2,1} = 0xe0cb4940, & N_{2,9}^{\star} = 0xa76e6485 \\
N_{2,2} = 0xba1d1a31, & N_{2,10} = 0x7631462d \\
N_{2,3} = 0x2f7ec2af, & N_{2,11} = 0x1da15155 \\
N_{2,4}^{\star} = 0x88630f64, & N_{2,12} = 0x0f4c9da9 \\
N_{2,5} = 0xebc021d8, & N_{2,13} = 0x319865f2 \\
N_{2,6} = 0x643aeb2, & N_{2,14} = 0x2a3dcee8 \\
N_{2,7} = 0xf25223f4, & N_{2,15} = 0x577c16c7
\end{array}$$

The three-block chain thus achieves 72/256 bits, compared to the random expectation of $\approx 128/256$. The three successive blocks reduce the near-collision weight as $95 \rightarrow 75 \rightarrow 72$, with diminishing but consistent gains at each stage.

Three-block chain under the (13, 4) differential. Running the same three-block procedure with the (13, 4) partial-dead differential—using the optimal block-2 message ($\|\Delta H_2\|_H = 74/256$, Section 7.3) as the intermediate IV—coordinate descent over random block-3 starting points (13 trials) independently reaches $\|\Delta H_3\|_H = \mathbf{72/256}$, matching the Nine-Step three-block minimum. The reduction chain is $137 \rightarrow 74 \rightarrow 72$. The block-2 words that changed from W_{B1} are: $N_{1,0} = 0xebe209e5$, $N_{1,1} = 0xae3c64d$, $N_{1,10} = 0x22f70a8a$. Complete CPU-verified block-3 message ($\star$ = changed from random start):

$$\begin{array}{ll}
N_{2,0}^{\star} = 0x0f79c1dc, & N_{2,8}^{\star} = 0xc6a01b18 \\
N_{2,1}^{\star} = 0x8867bb35, & N_{2,9}^{\star} = 0x40387c32 \\
N_{2,2}^{\star} = 0x3429c176, & N_{2,10}^{\star} = 0x5539ccbb \\
N_{2,3}^{\star} = 0x46c45627, & N_{2,11}^{\star} = 0x826ea374 \\
N_{2,4}^{\star} = 0x626b4377, & N_{2,12}^{\star} = 0x3fc8c606 \\
N_{2,5}^{\star} = 0x474524a4, & N_{2,13}^{\star} = 0x48d6c64b \\
N_{2,6}^{\star} = 0x293dae09, & N_{2,14}^{\star} = 0x8f75416b \\
N_{2,7}^{\star} = 0x6a68a515, & N_{2,15}^{\star} = 0x87c954d1
\end{array}$$

8 Comparison with Prior Work

^c *Weakened* problem: only a prefix of the 256-bit output is constrained (to zero), not the full target. ^d We have not obtained the full text and cannot confirm whether the target is full or, like ^c, partially constrained; the entry records what is reported, and we make no priority claim at 19 rounds either way (§8).

Table 7: Preimage results on reduced-round SHA-256. ^aPseudo-preimage; ^bconverted from pseudo-preimage. “Concrete” = at least one actual example found and verified.

Reference	Rounds	Work	Oracle-free	Concrete
Aoki–Sasaki [3]	24	$2^{52.3a} / 2^{104.5b}$	Yes	No
Pikhtovnikov–Kiryanova [6]	18	multi-day	Yes	Yes
Davydov et al. [8]	18	SAT, 2.38–20.6 s	Yes	Yes
Davydov et al. [8]	19	SAT, <i>unsolved</i>	Yes	No
Davydov et al. [8]	19 ^c	SAT + cube-and-conquer	Yes	Partial only
Zaikin [7] ^d	19	param. SAT + cube-and-conquer	Yes	Reported
This work	19	$O(2^{32})$	Yes	Yes (6 examples)

The MITM result of [3] achieves a lower round count in theory but has never been executed; the $2^{104.5}$ preimage work is far beyond practical reach.

The closest comparable practical results are the SAT-based line of [6, 8]. The most recent of these, [8], is worth describing in detail because it delimits the SAT frontier sharply. Using four independent CNF encodings (CBMC, Transalg, and SAT-encoding in Counter-chain, Dot-matrix, Two-operand and Espresso variants) with the Kissat solver at a 5000s limit, that work finds full-target preimages of the compression function at 16, 17 and 18 rounds—18 rounds in between 2.38 s and 20.6 s depending on encoding—and reports *not solved* at 19 rounds for *every* encoding tried.

Their “weakened 19-round” result is a different problem: rather than fixing the entire 256-bit output, only a zero prefix of it is constrained, which multiplies the number of admissible preimages. Plain Kissat reached a 24-bit zero prefix; a 24-hour cube-and-conquer run (EnCnC, 16 cores) solved every case except the 32-bit one, i.e. at most 31 of the 256 output bits. The authors conclude that finding a preimage for 19 rounds of the SHA-256 compression function is “too difficult a task even if one carefully selects the SAT encoding and uses parallel computing.”

The present work reports three full-target 19-round preimages, all 256 output bits fixed, whereas the weakened-19 row of [8] constrains at most 31/256 of the output. Zaikin [7] reports inverting the 19-round compression function by a parameterized Cube-and-Conquer solver; at the time of writing we have not obtained that paper’s full text and therefore make no priority claim at 19 rounds. What distinguishes the present work is the method—an algebraic reduction to table lookups with an $O(2^{32})$ per-context inner loop on a single GPU—rather than being first to the round count. We emphasise that this is a difference in kind rather than degree: a partially-constrained output is a near-preimage condition, not a preimage. Our technique also differs from that line in kind, being algebraic and $O(2^{32})$ per context attempt rather than search-based, which is why it succeeds where careful SAT encoding and parallel search do not.

An independent corroboration of where the SAT frontier lies: in our own experiments a modern bit-vector solver (Bitwuzla 0.9.1) solves the direct all-message preimage problem for this target through 18 rounds—R16 in 0.23 s, R17 in 1.25 s, R18 in 63.1 s with a 300 s cap—but returns UNKNOWN at 19 rounds under a 600 s cap and at 20 rounds under 60 s. That the generic-solver boundary falls between 18 and 19 rounds is consistent with 18 being the SAT-reachable limit, and indicates that the 19-round result here is not within reach of generic search.

Attack scope. The attack covers 19 of 64 rounds ($\approx 29.7\%$) and targets the compression function without padding constraints. Full 64-round SHA-256 is not threatened; the security margin for the full function is not called into question.

9 Conclusion

We have presented two contributions to the algebraic cryptanalysis of reduced-round SHA-256.

Preimage (19 rounds). Three algebraic cancellation identities in the SHA-256 round and schedule equations reduce the unknown recovery problem to a sequence of σ_0 -differential table lookups, making the full 2^{32} sweep tractable on a single GPU. Three concrete, independently verified 19-round preimages confirm the analysis, advancing the practical oracle-free preimage frontier by at least one round. The oracle-free attack on 20 rounds remains open; the fourth schedule constraint (C3) introduces an independent 2^{-32} filter, raising the expected contexts by a factor of 2^{32} beyond the $R = 19$ result (projected $\mu_{R=19} \approx 6 \times 10^{-3}$), a barrier for which no bypass is currently known. We do not claim it is fundamental: the decisive open question is whether the C0–C3 constraint graph admits an elimination order whose largest unresolved separator stays near 32 bits, rather than whether an additional round adds an additional equation.

The 20-round barrier. We characterise that barrier quantitatively in a companion paper [1], which shows the $R=19$ architecture transplants to $R=20$ in form but not in efficacy, inventories which absorptions admit a global table, and proves a new exact identity for σ_0 . Its results are negative and we do not restate them here.

Nine-Step structure (collision direction). The Nine-Step Cancellation Lemma establishes an 8-round algebraic dead zone in the SHA-256 message schedule under the differential ($\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$). This structure enables a GPU birthday sweep that locates, in 5.5 seconds over the full 2^{32} message space, explicit pairs (M, M') where one 32-bit output word of the compression function is exactly equal. Eight such pairs are enumerated and verified. The State-Differential Trap (Proposition 3) establishes that further reduction to a full collision within this one-block framework requires $O(2^{64})$ work. A systematic survey of all 240 two-position message-schedule differentials confirms that the Nine-Step pair $(0, 9)$ is the *unique* two-position differential achieving a full 8-round algebraic dead zone; however, partial-dead differentials achieve competitive improvements in the multi-block setting. Among eight surveyed differentials, the $(3, 12)$ pair achieves **72/256** in *two* blocks—matching the three-block Nine-Step minimum—with $(12, 3)$ at 73/256 and $(13, 4)$ at 74/256. A three-block Nine-Step chain also reaches **72/256** ($95 \rightarrow 75 \rightarrow 72$); independently, a three-block $(13, 4)$ chain reaches the same bound ($137 \rightarrow 74 \rightarrow 72$), confirming that 72/256 is a structural floor of the coordinate-descent method rather than an artefact of one differential. A four-block birthday sweep (varying the entire block-4 message word W_{15} over 2^{32} values with the three-block chain fixed) yielded a single-word minimum of 78/256; subsequent coordinate descent over all 16 block-4 words confirmed 78/256 as a deep local minimum (20 independent restarts, no improvement). Adding a fourth block thus does *not* improve below the three-block 72/256 floor, further supporting its structural character. The total improvement is $184/256 = 71.9\%$ from the all-different baseline. Exhaustive single- and multi-word sensitivity analysis confirms that the 75/256 two-block minimum is a ≥ 4 -bit-deep local minimum: no perturbation of fewer than 4 of the 512 message bits improves the Hamming weight, and coordinate descent on the block-1 message (reducing $\|\Delta H_1\|$ from 95 to 75/256) still yields 75/256 for block-2—confirming the minimum is a structural property of the nine-step chain, not an artifact of the block-1 choice. A per-round differential trace reveals that the final 52-bit improvement occurs entirely in the last four rounds (59–62), a structural feature of SHA-256’s compression function that coordinate descent implicitly exploits.

Artefacts. The solver code, table-build utility, and independent verifier are available in the accompanying reproducibility package at

<https://github.com/kamb-code/sha256-r19-preimage>. All three verified preimage examples (Table 2) can be checked with:

```
python3 code/verify_r19.py --rounds 19 --hash <hex> --words "<W0 ...W15>"
```

References

- [1] K. S. Basra. The 20-Round Barrier for Table-Based Preimage Attacks on SHA-256. Companion paper, 2026.
- [2] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. Technical Report, NIST, 2015.
- [3] K. Aoki, J. Guo, K. Matusiewicz, Y. Sasaki, and L. Wang. Preimages for step-reduced SHA-2. In *Advances in Cryptology – ASIACRYPT 2009*, Lecture Notes in Computer Science, vol. 5912, pp. 578–597. Springer, 2009.
- [4] I. Mironov and L. Zhang. Applications of SAT solvers to cryptanalysis of hash functions. In *Theory and Applications of Satisfiability Testing (SAT 2006)*, Lecture Notes in Computer Science, vol. 4121, pp. 102–115. Springer, 2006. [Collision attacks; no preimage results.]
- [5] S. K. Sanadhya and P. Sarkar. New collision attacks against up to 24-step SHA-2. In *Progress in Cryptology – INDOCRYPT 2008*, Lecture Notes in Computer Science, vol. 5365, pp. 91–103. Springer, 2008.
- [6] A. Pikhtovnikov and A. Kiryanova. Preimage attack on SHA-256 with logical cryptanalysis. *Proceedings of the Institute for System Programming of the Russian Academy of Sciences*, vol. 34, no. 1, 2022.
- [7] O. S. Zaikin. Preimage attacks on round-reduced MD5, SHA-1 and SHA-256 using a parameterized SAT solver. *Constraints*, January 2026.
- [8] V. V. Davydov, M. D. Pikhtovnikov, A. P. Kiryanova, and O. S. Zaikin. Analysis of the cryptographic strength of the SHA-256 hash function using the SAT approach. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, vol. 25, no. 3, pp. 428–437, 2025.
- [9] D. Wagner. The boomerang attack. In *Fast Software Encryption (FSE 1999)*, Lecture Notes in Computer Science, vol. 1636, pp. 156–170. Springer, 1999.
- [10] R. Li and Z. Sun. Practical collision attacks against 31-step SHA-256. In *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2024.